



SAXSProcessor: a modular Python library for SAXS data reduction and analysis

N. Ratel-Ramond
LPCNO - Toulouse, France

I. Pipeline overview

II. Main features

III. SAXS Analysis: Nematic order parameter determination

IV. Application to SWING data

DATA Handling

filereaders.py

Xeuss Pilatus 1M, SWING,
ID02,...

DATA Processing

SAXSProcessor.py

reference subtraction, caving,
advanced plot, extraction of
azimuthal and radial profiles

DATA Analysis

nematicordercalculator.py,
utilities.py, utilities_SWING.py,
correlationdistancecalculator.py

I. Pipeline overview

II. Main features

III. SAXS Analysis: Nematic order parameter determination

IV. Application to SWING data

II. Main Features

1. DATA Handling: filereaders.py - 1 Class / instrument

- **EdfFile**: also handles data acquired with lineEraser strategy (individual files should be provided)
- **h5File_SWING**: class to manipulate SWING data
Methods: **convert2edf**, **convert_SWING_mask**
- **h5File_ID02**: class to manipulate ID02 ESRF data
- other file formats can be added through the definition of a dedicated class

Each of these class contains internal methods to extract frame numbers and magnetic field values (ex: EdfFile, search number before 'mT' in samplename)

2. DATA Processing: SAXSProcessor.py

Initialization of class to handle a **single** file **(multiple frames are averaged)**:

```
SAXSPROcessor(  
    file : str  
        Path to data file  
    reference_file : str  
        Path to reference data file for background subtraction  
    k : float = 1  
        Multiplier factor for reference subtraction  
    autosubtract: bool = True  
        Automatic determination of k factor for reference subtraction  
    instrument : str  
        'ID02', 'SWING', or 'LGC'  
    binning : int  
        Downsampling factor  
    mask : str  
        Path to mask file designed (pyFAI - EDF format)  
)
```

II. Main Features

2. DATA Processing: methods

SAXSProcessor.determine k(

Parameters:

data : array

Sample data

ref_data : array

Reference data

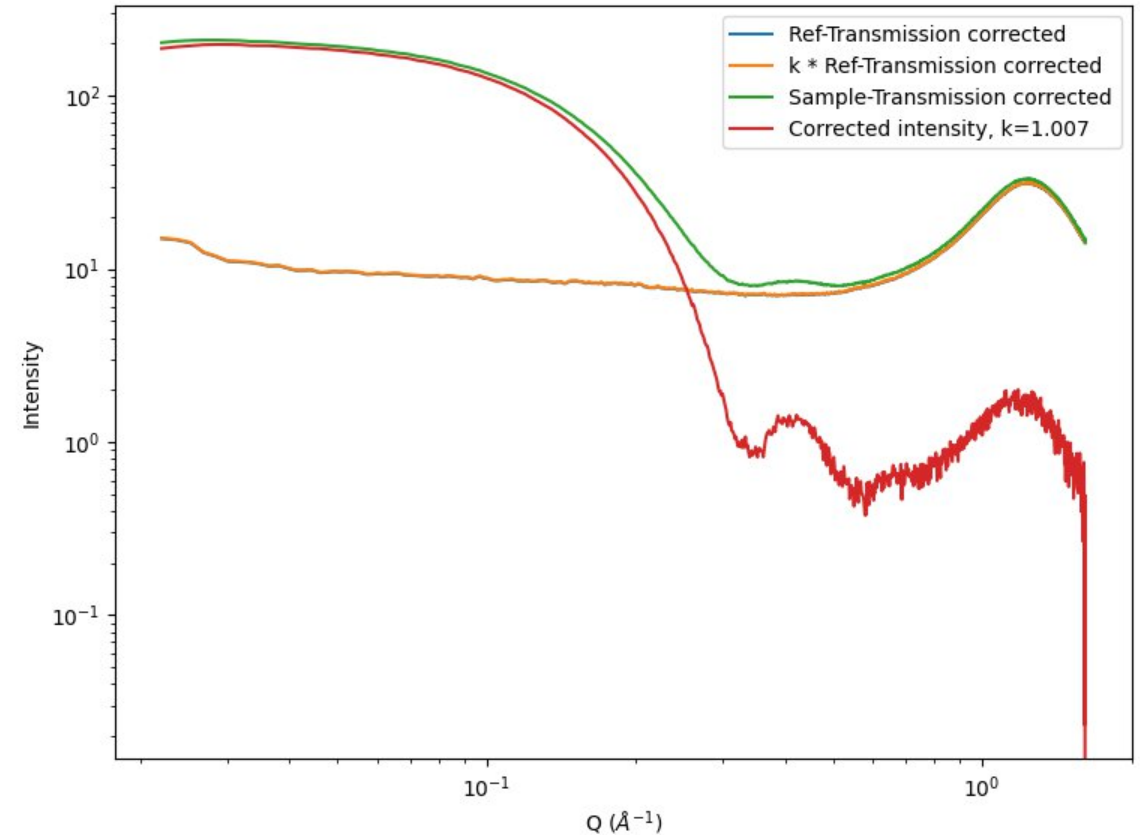
Returns:

k_opt : float

Optimal k factor

)

automatic determination of coefficient
for reference subtraction



fitting range: $q > 0.75 * q_{max}$

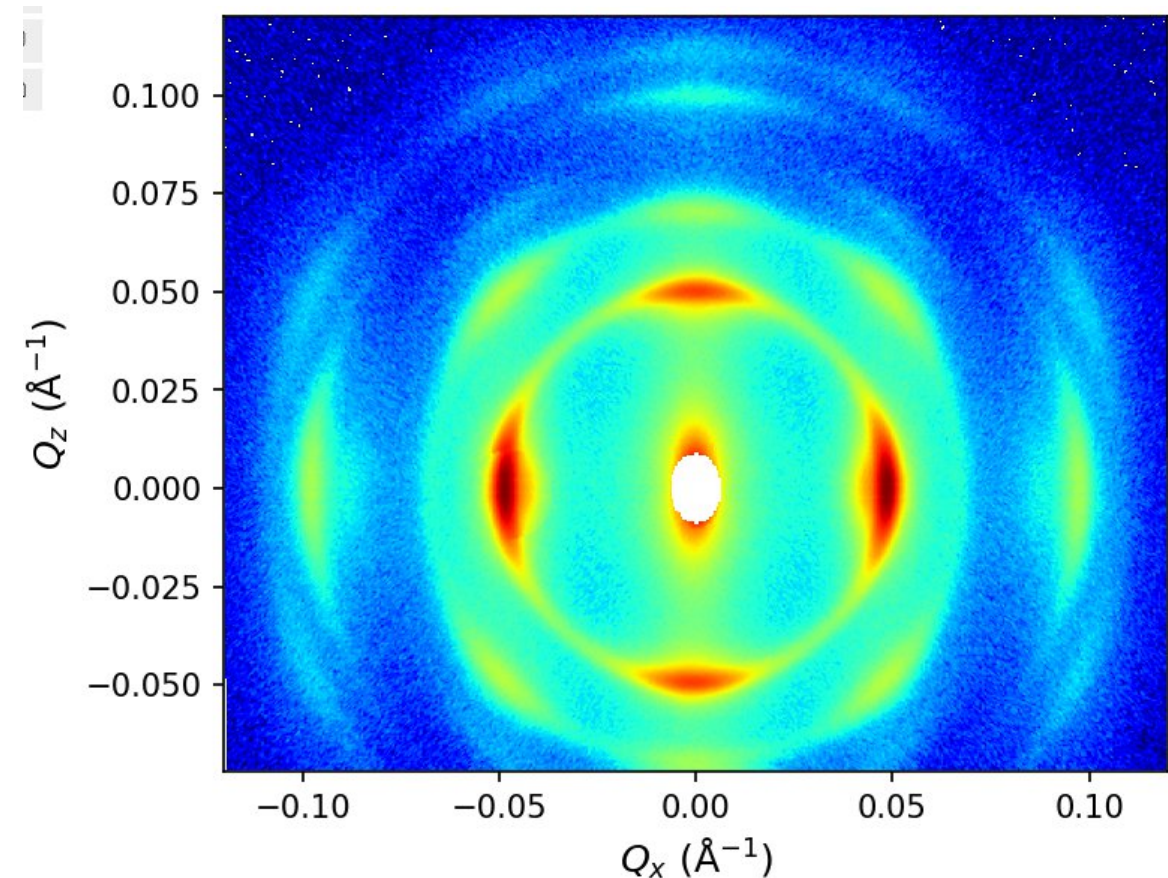
II. Main Features

2. DATA Processing: methods

`SAXSProcessor.apply_caving ()`

if a mask is provided, fill the gaps with
symetrically equivalent pixels (inversion
symmetry about beam center)

systematically applied upon initialization



cursor position in polar coordinates:

$q=0.0703 \text{ \AA}^{-1}$, $d=8.9368 \text{ nm}$, $\chi=45.6^\circ$

II. Main Features

2. DATA Processing: methods

SAXSProcessor.export_sasview (

Parameters

output_dir : str or None, optional

Directory where the SASView file will be written.

If None, a default directory ``<self.path>/sasview_exports/`` is used. The directory is created if it does not exist.

q_range : tuple(float, float) or None, optional

Tuple `(qmin, qmax)` defining the range of q values to export.

Notes

- Masked pixels (``self.maskdata == 1``) are written as NaN values.
- The data are flattened before export, as required by SASView for 2D data.
- The output filename follows the pattern:
``<output_dir>/<samplename>_<file_number>.dat``.

Output

A text file in ASCII format compatible with SASView 2D data import.

)

II. Main Features

2. DATA Processing: methods

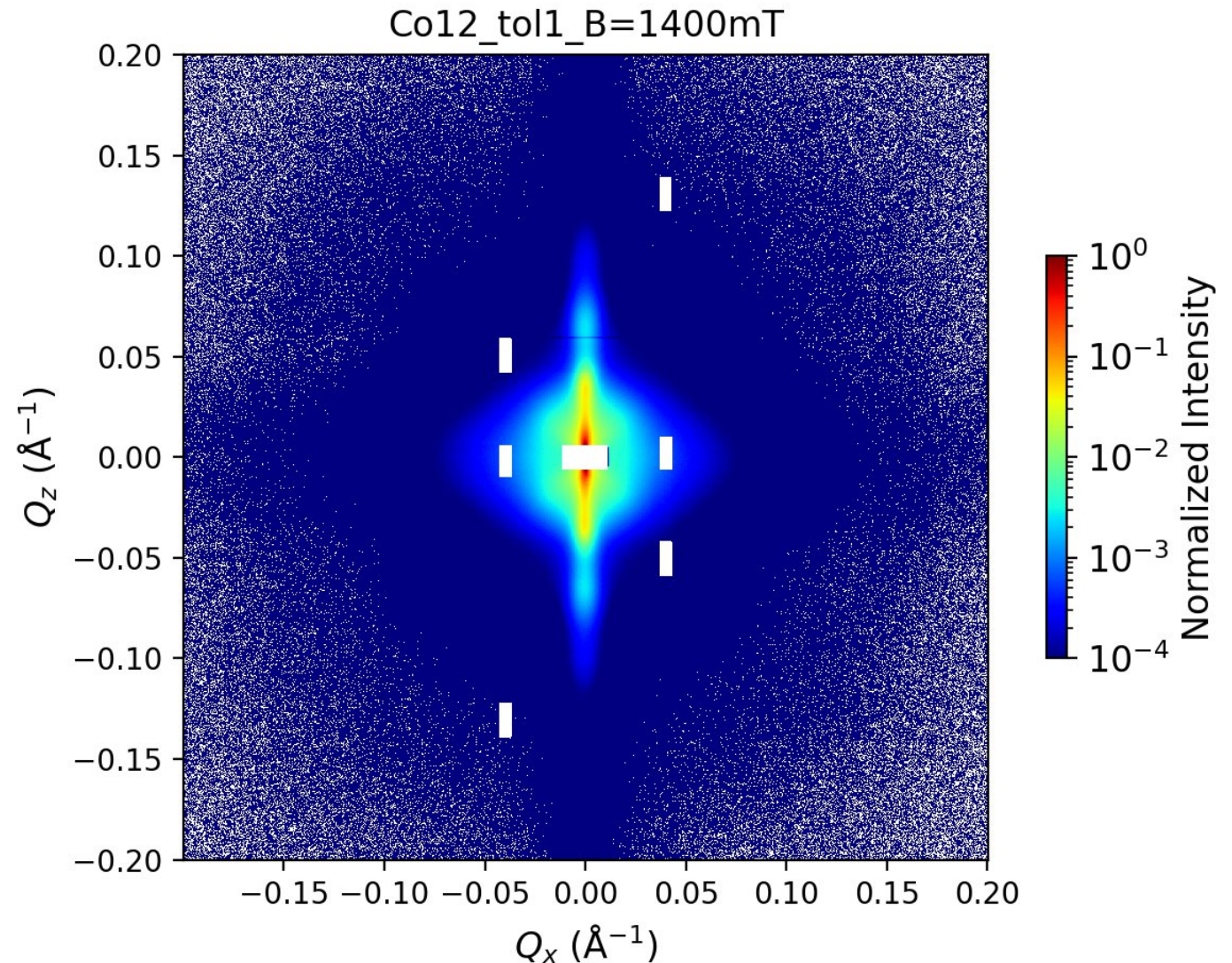
SAXSProcessor.plot2d_vsqs (

```

q_range : list = [0, 0.2]
    [qmin, qmax] range to display
cmap : str = 'jet'
    Colormap name
log : bool = True
    Use logarithmic scale
grid_size : int = 1000
    Interpolation grid size
vmin, vmax : float vmin=-4, vmax = 0
    Color scale limits
normalize : bool = True
    Normalize intensity
q_circles : list
    Q values for reference circle
output_dir : str
    Output directory for figure
rotate90 : bool = False
    Rotate image 90°

```

)



II. Main Features

2. DATA Processing: methods

SAXSProcessor.extract_azimuthal_profile(qvalue, threshold = 0.0001, save=True,output_dir=None):
Extract the azimuthal intensity profile at a given Q value using pyFAI.

Parameters

qvalue : float

Target Q value for the azimuthal integration ($\text{\AA}^{-1}$).

threshold : float, optional

Relative half-width of the radial integration window around `qvalue`
(default: 1e-4).

save : bool, optional

If True, save the azimuthal profile to a text file (default: True).

output_dir : str or None, optional

Base directory where the profile will be saved. If None, the profile
is saved in ``<self.path>/azimuthal_profiles/``.

Returns

chi : ndarray

Azimuthal angles in degrees.

I : ndarray

Azimuthally integrated intensity values.

II. Main Features

2. DATA Processing: methods

SAXSProcessor.find_main_orientation(self, qvalue=0.01, threshold=0.05):

"""

Determine the main orientation angle from the azimuthal intensity profile.

The azimuthal profile at a given Q value is smoothed and the angle corresponding to the maximum intensity is returned.

Parameters

qvalue : float, optional

Q value used for the azimuthal profile extraction.

threshold : float, optional

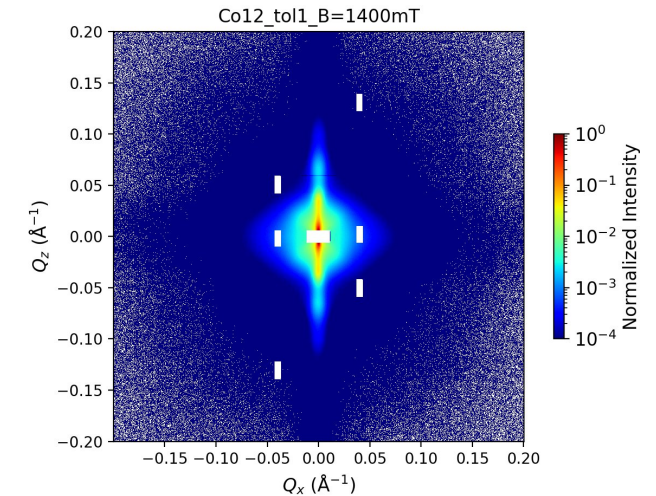
Relative width of the radial integration window around `qvalue`.

Returns

angle : float

Azimuthal angle (degrees) corresponding to the maximum intensity.

"""



main_orientation $\approx 0^\circ$

II. Main Features

2. DATA Processing: methods

SAXSProcessor.extract_radial_profile(azimuth: float = 90, width: float = 10, save=True, output_dir=None):

```
"""
Extract radial profile in angular sector.
```

```
Parameters:
```

```
-----
```

```
azimuth : float
```

```
    Central azimuthal angle (°)
```

```
width : float
```

```
    Angular sector full width (°)
```

```
save : bool
```

```
    Save profile to file
```

```
outputdir : str or None
```

```
    Directory to save profile
```

```
Returns:
```

```
-----
```

```
q : array
```

```
    Q values ( $\text{\AA}^{-1}$ )
```

```
I : array
```

```
    Intensity values
```

```
"""
```

Hint!

It may be useful to compute main_orientation before sector integration

I. Pipeline overview

II. Main features

III. SAXS Analysis: nematic order parameter determination

IV. Application to SWING data

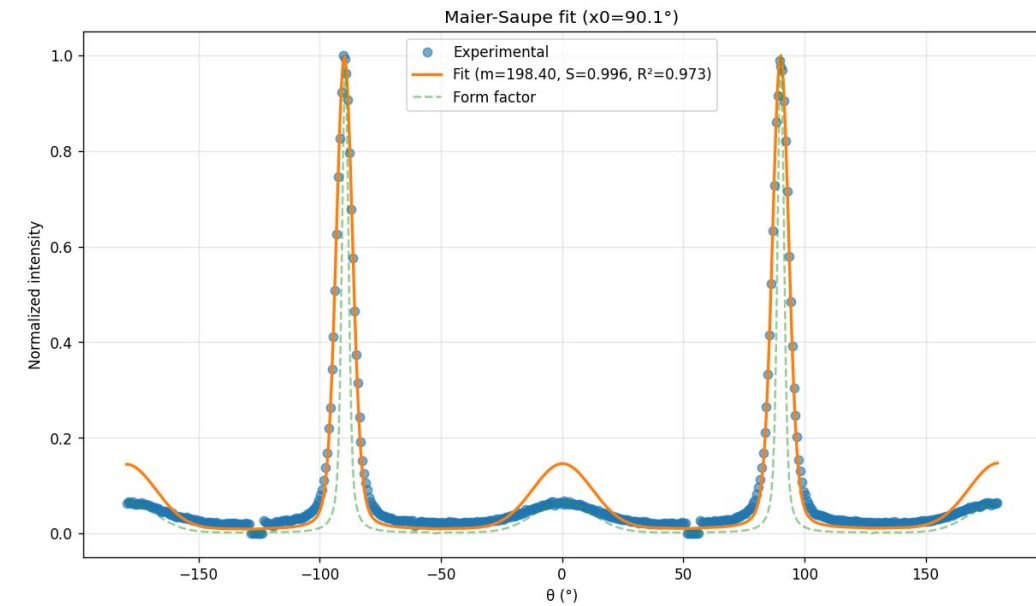
III. Nematic order parameter determination

- based on experimental azimuthal profile at constant q : $I_{exp}(\chi)$
 -> extraction with SAXSProcessor
- refinement of $I_{exp}(\chi)$ with the convolution of cylinder form factor and Maier Saupe distribution $f(\varphi)$

$$f(\varphi) = \frac{e^{m \cos^2(\varphi - x_0)}}{\int_0^\pi e^{m \cos^2(\varphi - x_0)} \sin \varphi d\varphi} \quad \text{and} \quad I_{exp}(\chi) \sim \int_0^{2\pi} f(\varphi) |F(q, \chi)|^2 d\varphi$$

- 2D form factor computation of cylinder (orientation φ)
 -> sasmodels (sasview)
- extraction of azimuthal profile associated to form factor
 -> $F(q, \chi)$
- refinement
- computation of nematic order parameter

$$S = \langle P_2(\cos \varphi) \rangle = \frac{\int_0^\pi \frac{1}{2} (3 \cos^2(\varphi - x_0) - 1) e^{m \cos^2(\varphi - x_0)} d\varphi}{\int_0^\pi e^{m \cos^2(\varphi - x_0)} \sin \varphi d\varphi}$$



III. Nematic order parameter determination

Implementation in `nematicordercalculator.py`:

3 classes: - `CylinderFormFactor`

```
processor : SAXSProcessor
    SAXS data processor (detector geometry inherited if provided)
npix1, npix2 : int
    Detector pixels (required if processor not provided)
pix_size : float
    Pixel size in m (required if processor not provided)
distance : float
    Sample-detector distance in m (required if processor not provided)
wl : float
    Wavelength in Å (required if processor not provided)
radius , L : float
    Cylinder radius and length (Å)
theta, phi : float
    Orientation angles (°)
radius_pd, L_pd : float
    Polydispersity
```


III. Nematic order parameter determination

Implementation in nematicordercalculator.py:

3 classes: - NematicOrderCalculator (CylinderFormFactor)

- BatchNematic: Process multiple SAXS files and extract nematic order parameters.

path : str,	Directory containing data files
instrument : str,	Beamline name
qvalues : list,	Q values for azimuthal profiles ($\text{\AA}^{-1}$)
file_filter : str,	Wildcard pattern for files
threshold : float,	Q tolerance for azimuthal profile extraction
binning : int,	Binning factor
mask : str,	Path to mask file
reference_file : str,	Path to reference file for background subtraction
k : float,	Scaling factor for background subtraction
autosubtract : bool,	Enable automatic background subtraction
form_factor : CylinderFormFactor or None	Form factor object (profiles extracted on-the-fly for each q)
output_dir : str,	Directory to save output files
structure:	CrystalStructure instance for WAXS data analysis



creation of .csv file with fitting results

III. Nematic order parameter determination

Function in utilities.py:

```
compute_nematic_parameter(
    processor = None,
    qvalue = 0.034,
    threshold = 0.05,
    radius = 78,
    L = 840,
    radius_pd = 0.3,
    L_pd = 0.75,
    plot=False,
    apply_mirror=None,
    verbose=True
)
```



Performs the following steps:

1. extract azimuthal profile
processor.Extract_azimuthal_profile(qvalue,threshold)
2. determine main orientation
processor.find_main_orientation()
3. compute corresponding form factor
CylinderFormFactor(radius,L,radius_pd,L_pd)
4. calculates nematic order parameter

returns S, main_orientation, fitting params (dict)

I. Pipeline overview

II. Main features

III. SAXS Analysis: determination of correlation distance

IV. Application to SWING data

III. Determination of correlation distance

3 methods implemented in `correlationdistancecalculator.py`

- 1 method 'derivative' based on `scipy.find_peaks` from the second derivative of SAXS signal
- 1 method 'spv' based on base line removal (power-law), and peak refinement using split pseudo voigt profile functions
- 1 method 'hybrid' based on the second derivative for peak identification and split pseudo peak refinement for the determination of correlation distance

Each method has specific args, all wrapped up in the same function `compute_correlation_distances()`

III. Determination of correlation distance

```
compute_correlation_distances(  
    nb_peaks=1,  
    azimuth: float = 90,  
    width: float = 40,  
    method='hybrid',  
    plot=True,  
    **detection_params)
```

detection params depend on selected method (see next slide)

This function is designed to compute correlation distances in specific azimuthal directions (or entire diagram if width = 360)

III. Determination of correlation distance

TABLEAU RÉCAPITULATIF DES PARAMÈTRES
 ✓ INTERFACE UNIFORMISÉE

Paramètre	Méthode	Défaut	Description
qmin	Both ✓	None	q min ($\text{\AA}^{-1}$)
qmax	Both ✓	None	q max ($\text{\AA}^{-1}$)
plot	Both ✓	False	Graphiques
subtract_power_law	SPV	True	Soustraire $I \sim q^{-m}$
power_law_method	SPV	'cancel'	'cancel' ou 'feat'
power_law_order	SPV	None	m fixe ou None=auto
power_law_range	SPV	(2.5,4.0)	Plage pour m
smooth	SPV	False	Lissage gaussien
smooth_sigma	SPV	2	Paramètre lissage
verbose	SPV	True	Infos détaillées
window_length	Deriv	15	Fenêtre S-G (impair)
polyorder	Deriv	3	Ordre polynomial
prominence	Deriv	0.5	Prominence min
distance_pts	Deriv	20	Distance min (pts)

Hybrid method (not described in the table) is the default method.

It accepts all the arguments described in the table + 1 arg (fit_window_width)

Most relevant args to play with are:

- **nb_peaks,**
- **qmin,**
- **qmax,**
- **fit_window_width (float)**
- **subtract_power_law(bool)**

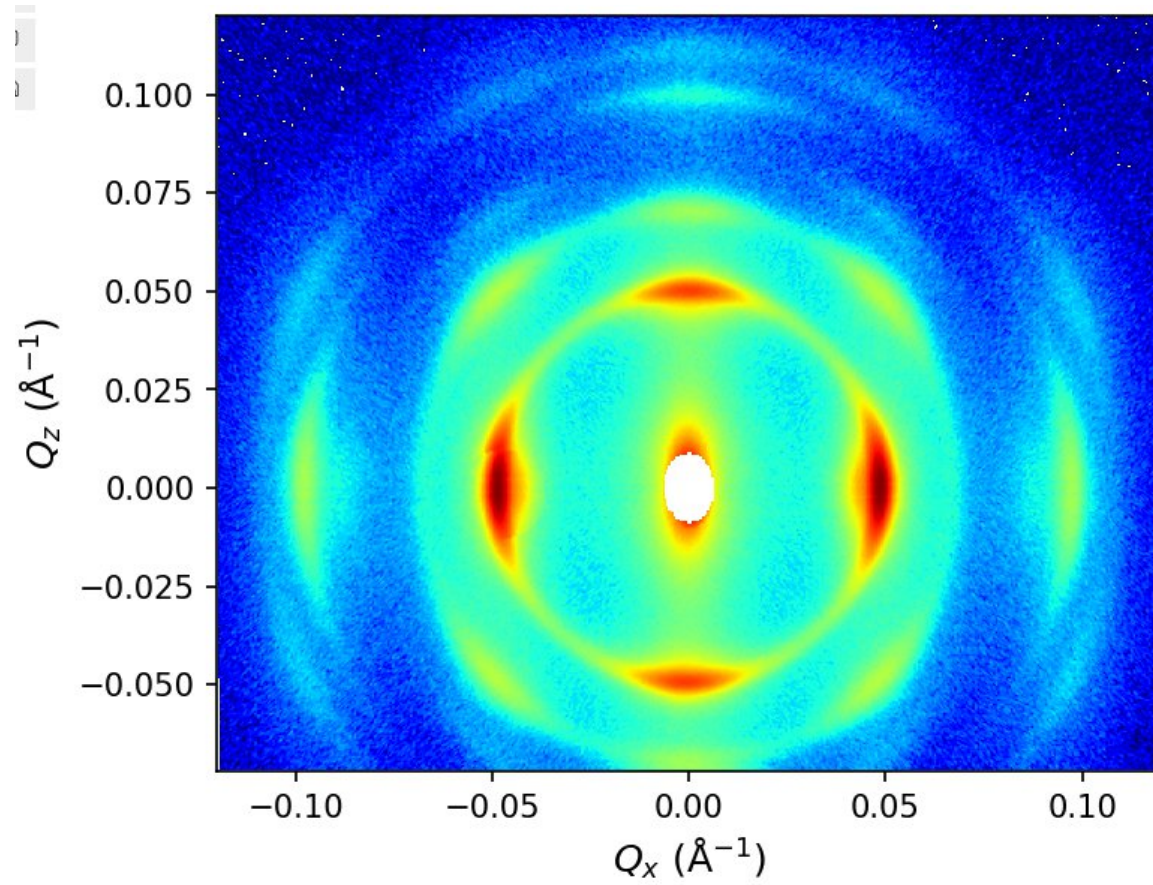
qmin and qmax must be chosen with care.

No panic! All the args have a default value (no impact if you miss one)

They are also filtered according to the method used (no error if you pass an arg unused by the method)

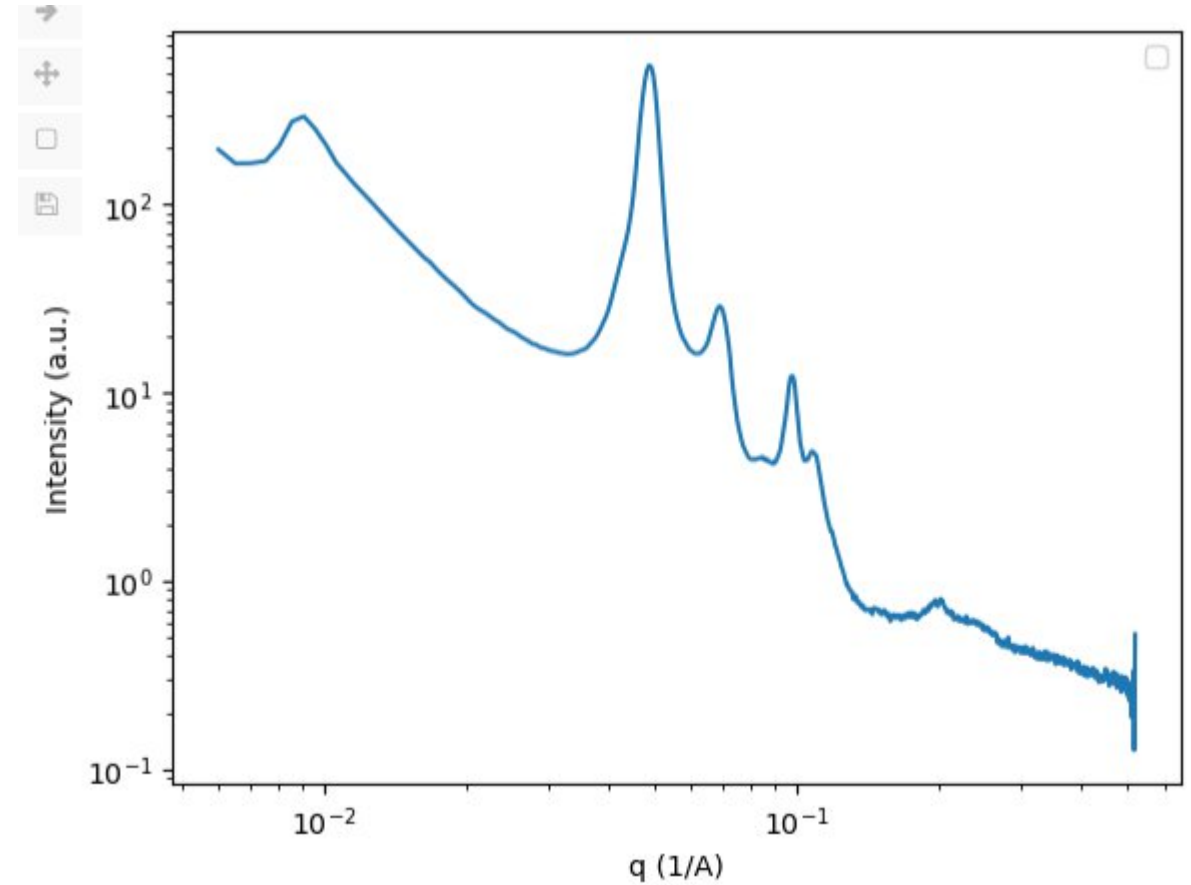
III. Determination of correlation distance

Self assembly of Fe NPs under magnetic field



$q=0.0703 \text{ \AA}^{-1}$, $d=8.9368 \text{ nm}$, $\chi=45.6^\circ$

Radial profile



III. Determination of correlation distance

method = 'derivative'

```
corr= CorrelationDistanceCalculator(proc)
```

```
# Second derivative method (no uncertainties computed - less accurate)
```

```
distances_deriv, err, _ = corr.compute_correlation_distances(
```

```
    nb_peaks=6,
```

```
    azimuth=90,
```

```
    width=360,
```

```
    plot=True,
```

```
    qmax=0.125,
```

```
    qmin=0.03,
```

```
    # method specific parameters
```

```
    method='derivative', # or 'derivative'
```

```
    window_length=15, # savitzky-golay filter window length
```

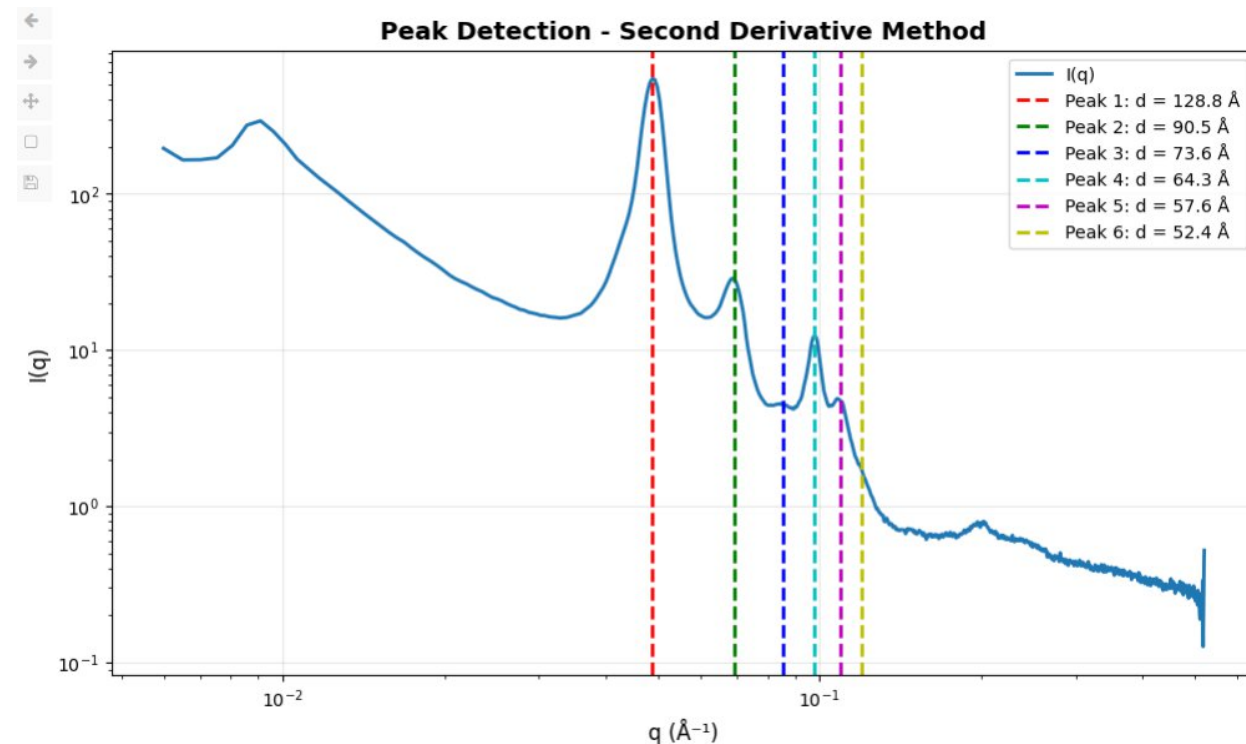
```
    polyorder=3, # savitzky-golay filter polynomial order
```

```
    prominence=0.5, # peak prominence (scipy find_peaks)
```

```
    distance_pts=20 # minimal distance between peaks in
```

```
number of points
```

```
)
```

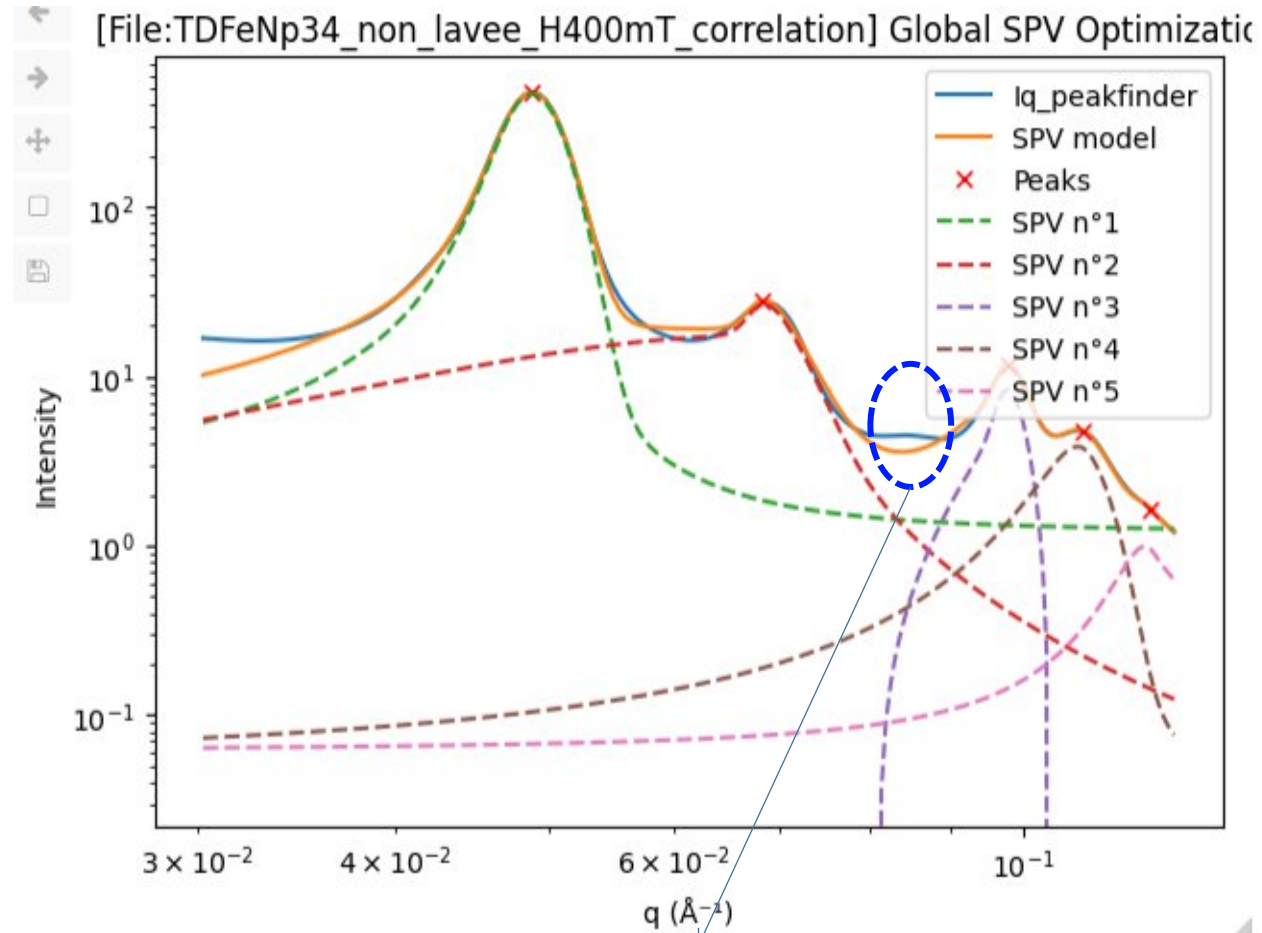


peaks found: 6/6
position not very accurate

III. Determination of correlation distance

method = 'spv'

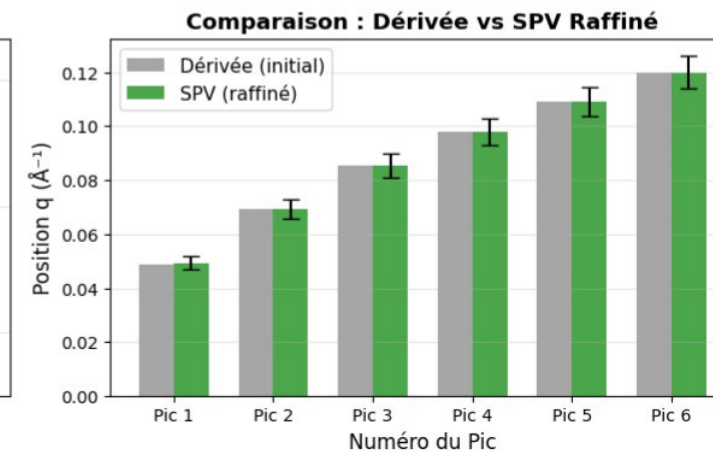
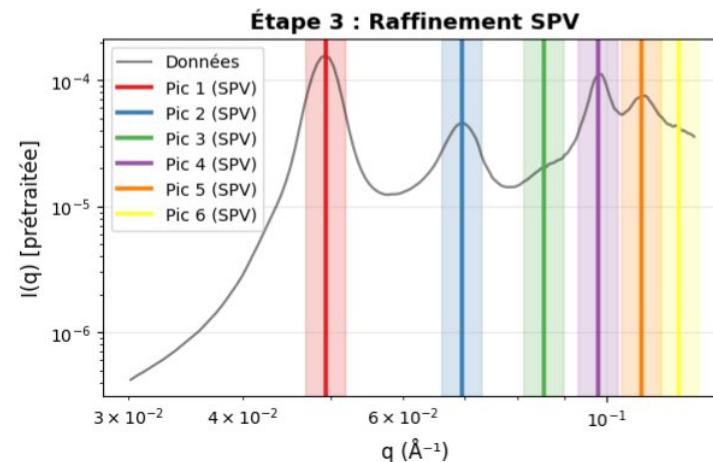
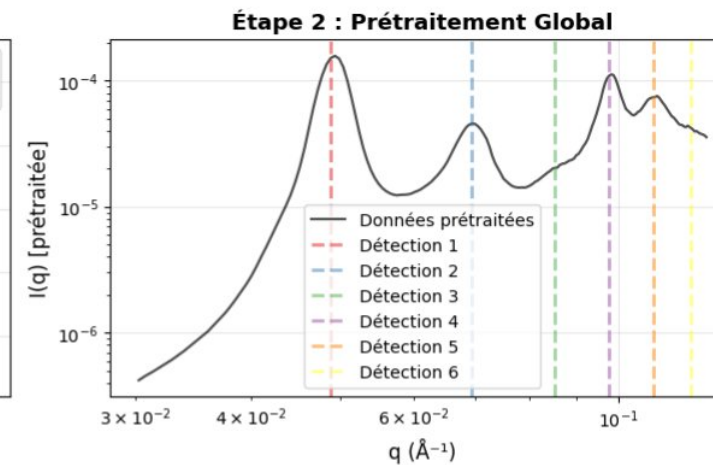
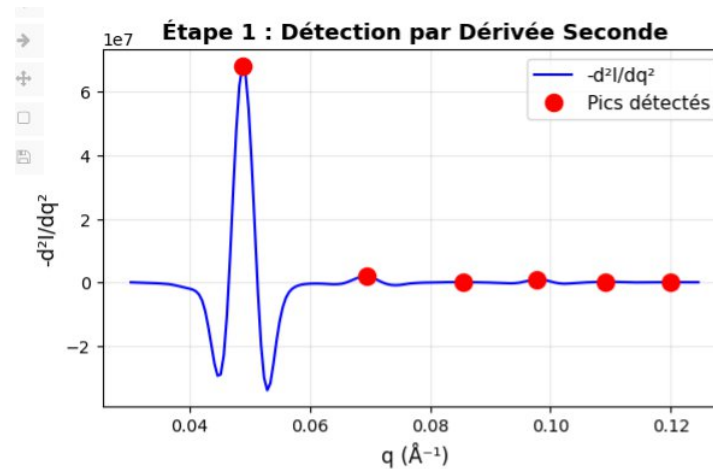
```
distances_spv, distances_std_spv, _ =
corr.compute_correlation_distances(
    nb_peaks=6,
    azimuth=90,
    width=360,
    plot=True,
    qmax=0.125,
    qmin=0.03,
    verbose=False,
    # method specific parameters
    method='spv',
    subtract_power_law=False, # subtract power law
background
    power_law_method='cancel', # 'feat' or 'cancel'
    power_law_order=None, # order of power law to subtract
(None for automatic)
    power_law_range=(2.5, 5.0), # q-range for power law fitting
    smooth = True, # apply gaussian smoothing to the data
    smooth_sigma = 2 # sigma for gaussian smoothing
)
```



III. Determination of correlation distance

method = 'hybrid'

```
distances_hybrid, distances_std_hybrid, _ = corr.compute_correlation_distances(
    nb_peaks=6,
    azimuth=90,
    width=360,
    method='hybrid',
    qmin=0.03,
    qmax=0.125,
    # Détection initiale (dérivée)
    window_length=15,
    polyorder=3,
    prominence=0.5,
    distance_pts=20,
    # Raffinement SPV
    subtract_power_law=True,
    power_law_method='cancel',
    power_law_order=None,
    power_law_range=(2.5, 5.0),
    smooth=False,
    smooth_sigma=2,
    fit_window_width=1,
    verbose=False,
    plot=True)
```



peaks found 6/6
+ accurate position

I. Pipeline overview

II. Main features

III. SAXS Analysis: nematic order parameter determination

IV. Application to SWING data

IV. Application to SWING μ SAXS data

Specificity:

1 .h5 file -> n frames -> linescan at a given z for n x positions



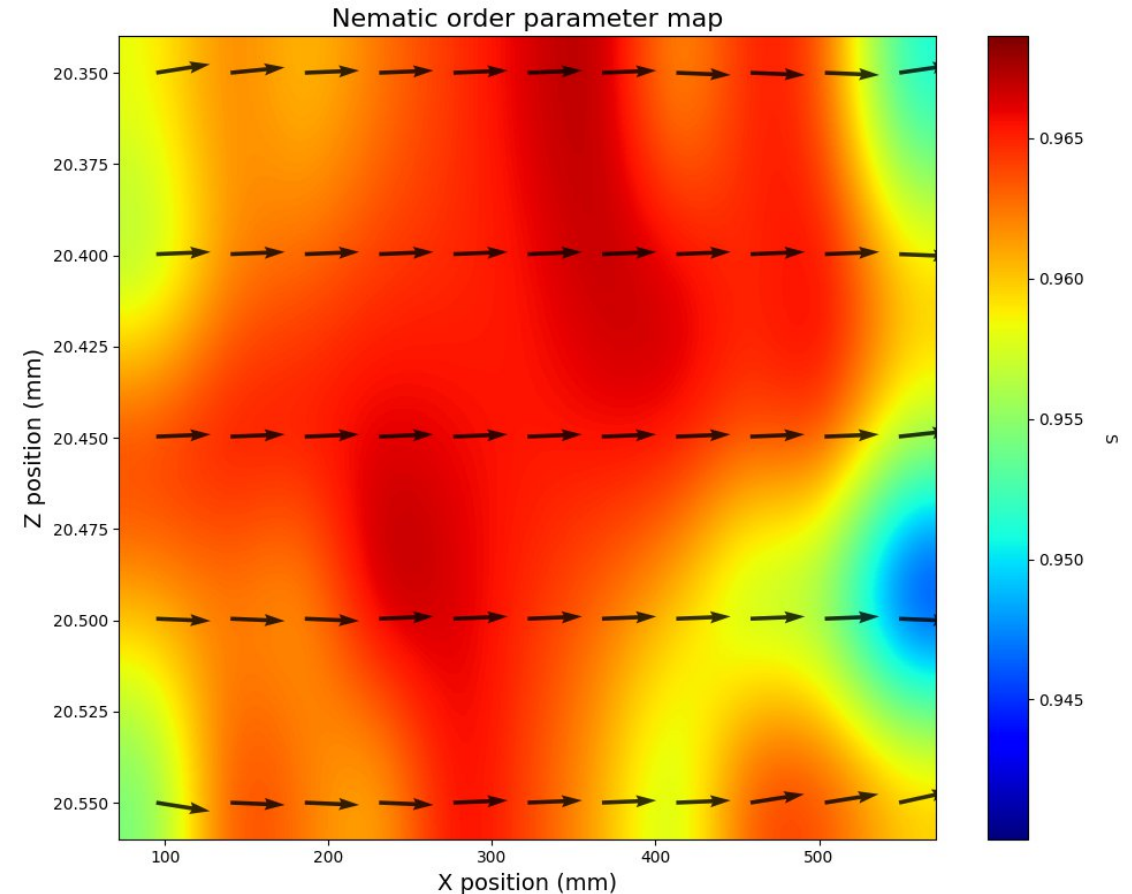
no averaging!

Specific workflow:

1. Convert and save individual frames to edf
2. Single frame analysis (instrument = 'LGC')

Workflow implemented in utilities_SWING.py ->
compute_nematic_order_assembly_SWING

```
h5path,
prefix='lacroix',
reference_file: str = None,
k =1,
autosubtract: bool = True,
mask=None,
qvalue=0.034,
threshold=0.05,
radius=78,
L=840,
radius_pd=0.3,
L_pd=0.75,
plot=False,
apply_mirror=False,
verbose=True)
```



IV. Application to SWING μ SAXS data

compute_global_nematic_parameter

```
h5path,
prefix='lacroix',
reference_file: str = None,
k =1,
autosubtract: bool = True,
mask=None,
qvalue=0.034,
threshold=0.05,
radius=78,
L=840,
radius_pd=0.3,
L_pd=0.75,
plot=False,
apply_mirror=False,
verbose=True)
```



averaging of all frames to
compute a single S value

compute_nematic_parameter_linescan_SWING

```
h5path,
prefix='lacroix',
reference_file: str = None,
k =1,
autosubtract: bool = True,
mask=None,
qvalue=0.034,
threshold=0.05,
radius=78,
L=840,
radius_pd=0.3,
L_pd=0.75,
plot=False,
apply_mirror=False,
verbose=True)
```

